D20A / 37

Experiment No: 7

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory:

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Steps to Connecat Ganache with Metamask and Remix IDE:

1. Install Ganache:
    - Download and install Ganache from the Truffle Suite website.
    - Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
    - Open Metamask and switch to a Custom RPC Network.
    - Enter the Ganache RPC server URL (e.g., HTTP://127.0.0.1:7545).
    - Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
    - Open Remix IDE in the browser.
      Choose Injected Web3 as the environment in the “Deploy & Run Transactions” tab.
    - Remix will automatically connect to the Metamask account (linked with Ganache).
4. Create a Simple Solidity Smart Contract:
    - Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.).

```
pragma solidity ^0.8.0;
    contract SimpleStorage {
      uint public data;
      function set(uint x) public {
        data = x;
      }
```

5. Compile and Deploy the Contract:
   Compile using Remix.
   - Deploy via the Metamask account connected to Ganache.
   - Approve the transaction in Metamask.
6. Check Transaction Details on Ganache:
   - View mined transactions, block details, and balances in the Ganache interface.
7. Interact with the Smart Contract:
   - Use Remix to call contract functions (set() or get()).
   - Observe the state changes and corresponding transactions on Ganache.

Outcome**:**

- Understood how to link **Metamask** with **Remix IDE** for Ethereum smart contract deployment.
- Successfully deployed and interacted with a Solidity smart contract through **Ganache**.
- Verified all transactions and balances within the Ganache environment.

Implementation:

```
 // SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract ProductTraceability {

    address public admin;

    enum Role {
        None,
        Manufacturer,
        Wholesaler,
        Distributor,
        Retailer,
        Customer
    }

    mapping(address => Role) public roles;

    struct Product {
        uint256 id;
        string name;
        string manufacturer;
        address currentOwner;
        uint256 registeredAt;
        bool exists;
```

```
}

struct TransferRecord {
    address previousOwner;
    address newOwner;
    uint256 timestamp;
}

mapping(uint256 => Product) private products;
mapping(uint256 => TransferRecord[]) private history;

event RoleAssigned(address indexed account, Role role);
event ProductRegistered(
    uint256 indexed productId,
    address indexed manufacturer,
    string name
);
event OwnershipTransferred(
    uint256 indexed productId,
    address indexed previousOwner,
    address indexed newOwner,
    uint256 timestamp
);

constructor() {
```

```
    admin = msg.sender;
}

modifier onlyAdmin() {
    require(msg.sender == admin, "Only admin");
    _;
}

function assignRole(
    address account,
    Role role
) external onlyAdmin {
    require(account != address(0), "Invalid account");
    require(role != Role.None, "Invalid role");

    roles[account] = role;
    emit RoleAssigned(account, role);
}

function registerProduct(
    uint256 productId,
    string calldata name,
    string calldata manufacturer
) external {
    require(
```

```
    roles[msg.sender] == Role.Manufacturer,
    "Only manufacturer"
);
require(productId != 0, "Invalid product ID");
require(!products[productId].exists, "Product already exists");
require(bytes(name).length > 0, "Name required");
require(bytes(manufacturer).length > 0, "Manufacturer required");

products[productId] = Product({
    id: productId,
    name: name,
    manufacturer: manufacturer,
    currentOwner: msg.sender,
    registeredAt: block.timestamp,
    exists: true
});

history[productId].push(
    TransferRecord({
        previousOwner: address(0),
        newOwner: msg.sender,
        timestamp: block.timestamp
    })
);
```

```
    emit ProductRegistered(productId, msg.sender, name);
}

function transferOwnership(
    uint256 productId,
    address newOwner
) external {
    require(products[productId].exists, "Product does not exist");
    require(
        products[productId].currentOwner == msg.sender,
        "Not current owner"
    );
    require(newOwner != address(0), "Invalid new owner");
    require(newOwner != msg.sender, "Already owner");

    Role senderRole = roles[msg.sender];
    Role receiverRole = roles[newOwner];

    require(
        (senderRole == Role.Manufacturer &&
            receiverRole == Role.Wholesaler) ||
        (senderRole == Role.Wholesaler &&
            receiverRole == Role.Distributor) ||
        (senderRole == Role.Distributor &&
            receiverRole == Role.Retailer) ||
```

```
            (senderRole == Role.Retailer &&
                receiverRole == Role.Customer),
            "Invalid supply chain transfer"
        );

        products[productId].currentOwner = newOwner;

        history[productId].push(
            TransferRecord({
                previousOwner: msg.sender,
                newOwner: newOwner,
                timestamp: block.timestamp
            })
        );

        emit OwnershipTransferred(
            productId,
            msg.sender,
            newOwner,
            block.timestamp
        );
    }

    function verifyProduct(
        uint256 productId
```

```
) external view returns (bool) {
    return products[productId].exists;
}

function getProduct(
    uint256 productId
)
    external
    view
    returns (
        uint256 id,
        string memory name,
        string memory manufacturer,
        address currentOwner,
        uint256 registeredAt
    )
{
    require(products[productId].exists, "Product does not exist");

    Product storage p = products[productId];

    return (
        p.id,
        p.name,
        p.manufacturer,
```

```
        p.currentOwner,
        p.registeredAt
    );
}

function getTransferCount(
    uint256 productId
) external view returns (uint256) {
    require(products[productId].exists, "Product does not exist");
    return history[productId].length;
}

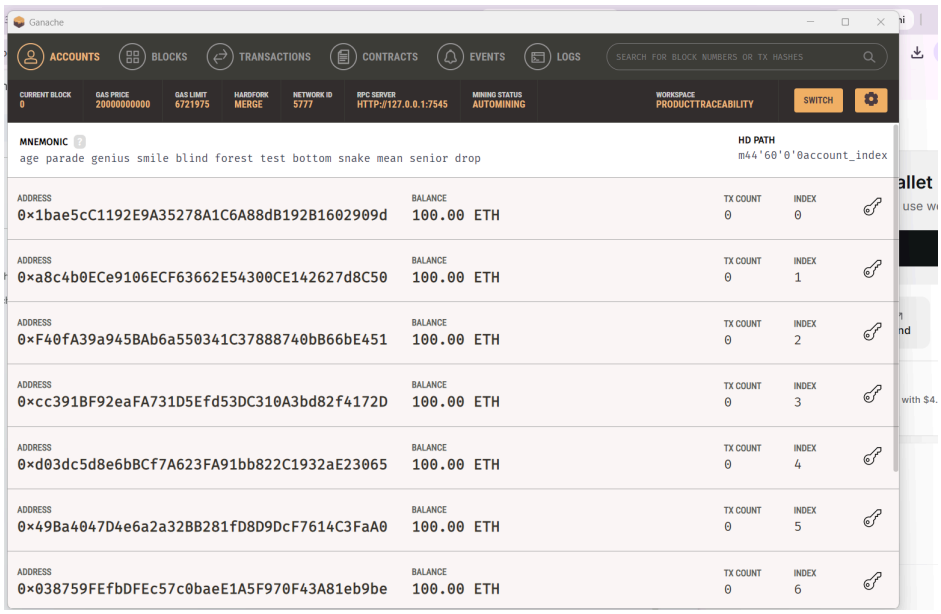
function getTransferRecord(
    uint256 productId,
    uint256 index
)
    external
    view
    returns (
        address previousOwner,
        address newOwner,
        uint256 timestamp
    )
{
    require(products[productId].exists, "Product does not exist");
```

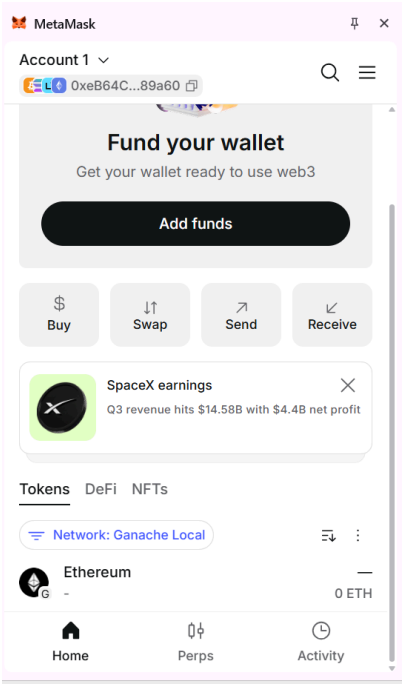
```
        require(index < history[productId].length, "Invalid index");

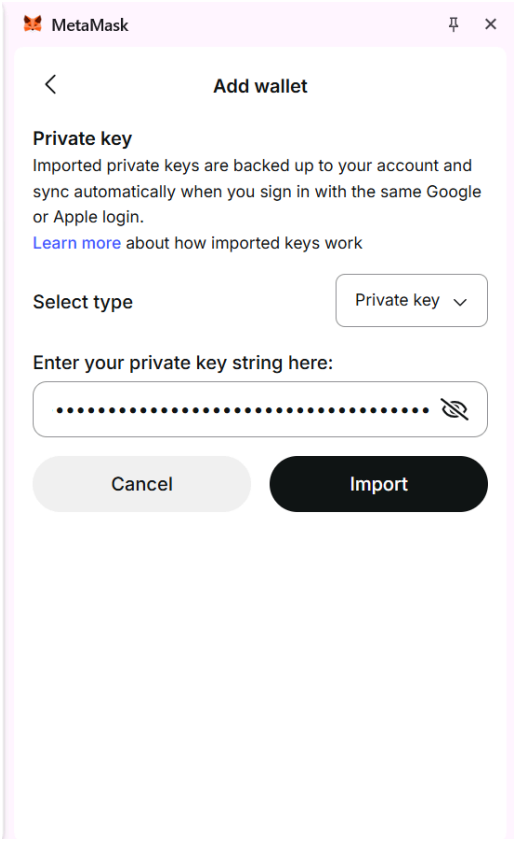
        TransferRecord storage record = history[productId][index];

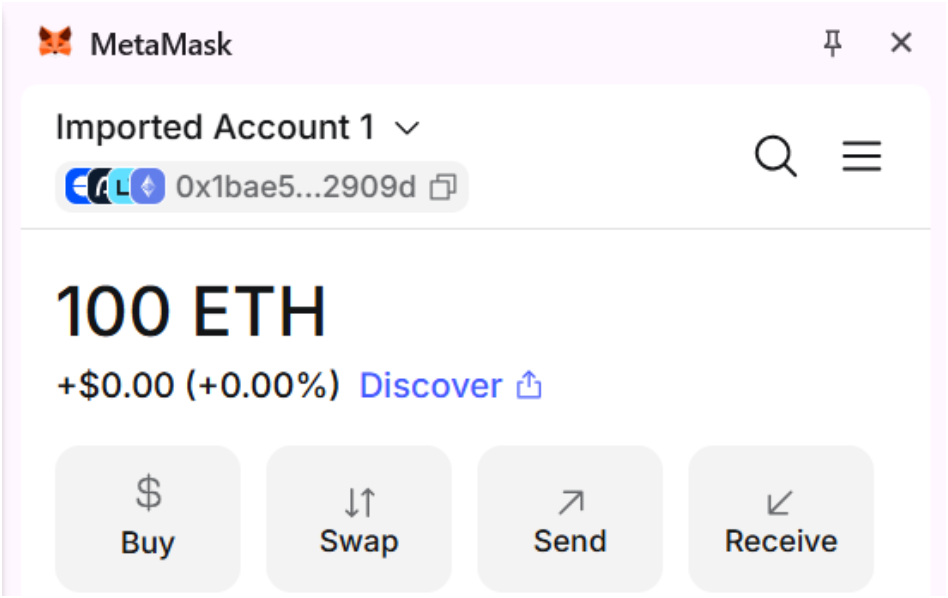
        return (
            record.previousOwner,
            record.newOwner,
            record.timestamp
        );
    }
}
```

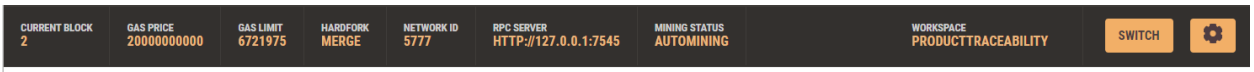
Output:

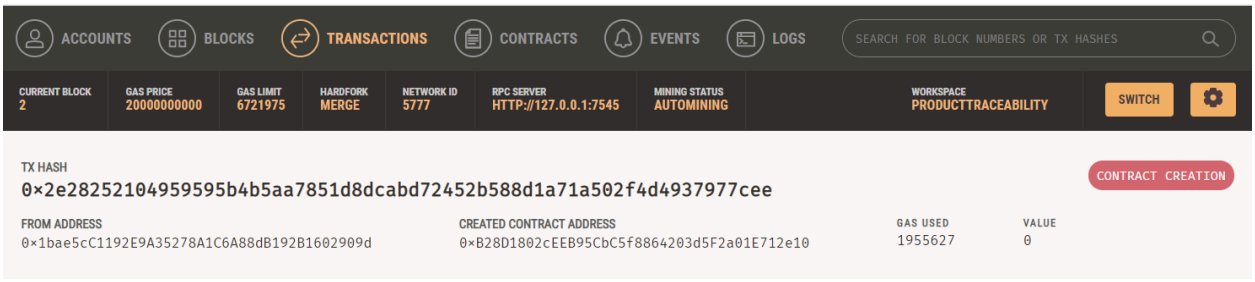


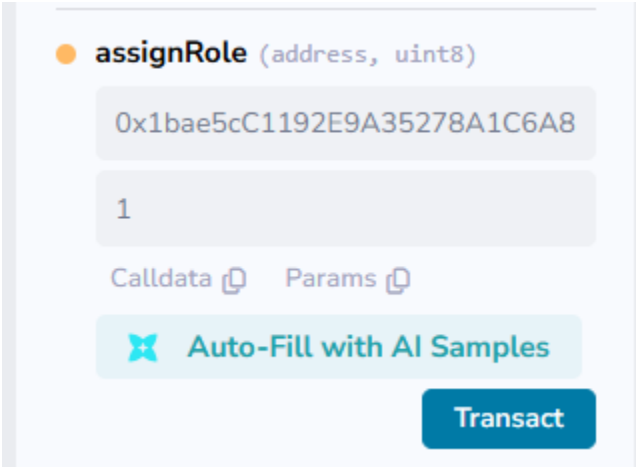

MetaMask
Account 1
0xeB64C...89a60
Fund your wallet
Get your wallet ready to use web3
Add funds
Buy
Swap
Send
Receive
SpaceX earnings
Q3 revenue hits $14.58B with $4.4B net profit
Tokens
DeFi
NFTs
Network: Ganache Local
Ethereum
-
0 ETH
Home
Perps
Activity

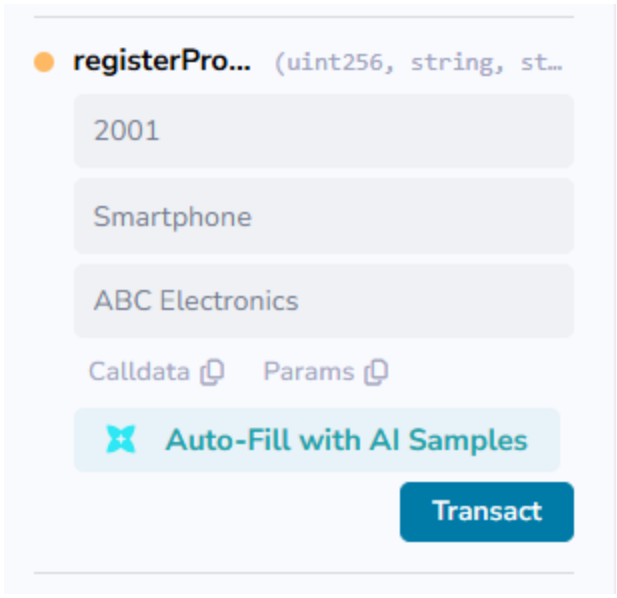

MetaMask
Add wallet
Private key
Imported private keys are backed up to your account and sync automatically when you sign in with the same Google or Apple login.
Learn more about how imported keys work
Select type
Private key
Enter your private key string here:
Cancel
Import


MetaMask
Imported Account 1
0x1bae5...2909d
100 ETH
+$0.00 (+0.00%) Discover
Buy
Swap
Send
Receive

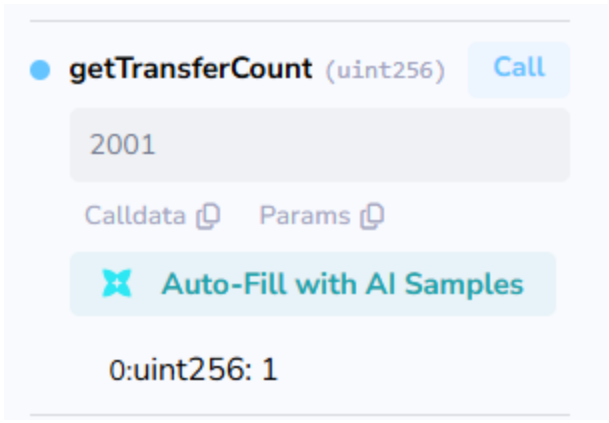

CURRENT BLOCK 2
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE PRODUCTTRACEABILITY
SWITCH

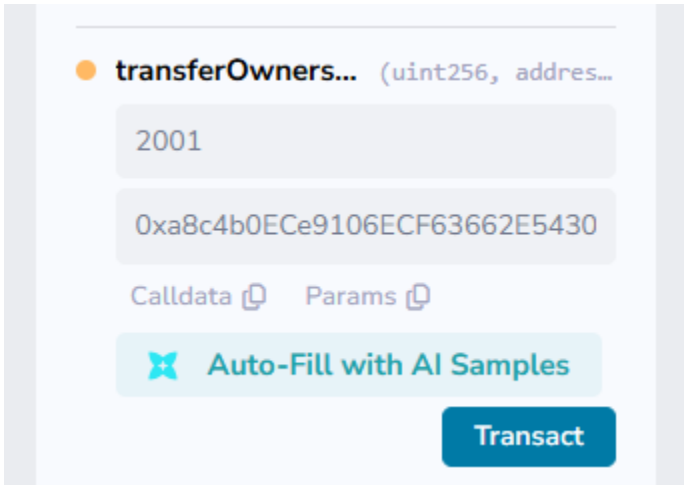

ACCOUNTS
BLOCKS
TRANSACTIONS
CONTRACTS
EVENTS
LOGS
SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 2
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE PRODUCTTRACEABILITY
SWITCH
TX HASH
0x2e28252104959595b4b5aa7851d8dcabd72452b588d1a71a502f4d4937977cee
CONTRACT CREATION
FROM ADDRESS
0x1bae5cC1192E9A35278A1C6A88dB192B1602909d
CREATED CONTRACT ADDRESS
0xB28D1802cEEB95CbC5f8864203d5F2a01E712e10
GAS USED
1955627
VALUE
0

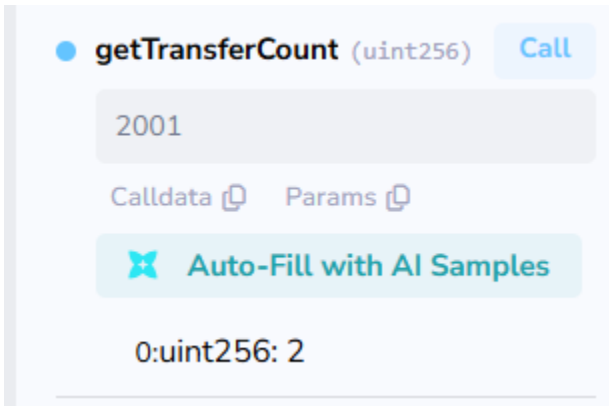

assignRole (address, uint8)
0x1bae5cC1192E9A35278A1C6A8
1
Calldata
Params
Auto-Fill with AI Samples
Transact


registerPro... (uint256, string, st...
2001
Smartphone
ABC Electronics
Calldata
Params
Auto-Fill with AI Samples
Transact

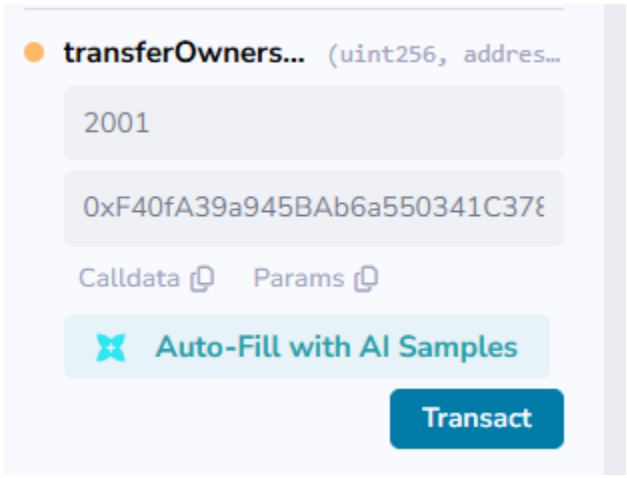

getProduct (uint256)
Call
2001
Calldata
Params
Auto-Fill with AI Samples
0:uint256: id 2001
1:string: name Smartphone
2:string: manufacturer ABC E lectronics
3:address: currentOwner 0x 1bae5cC1192E9A35278A 1C6A88dB192B1602909 d
4:uint256: registeredAt 179

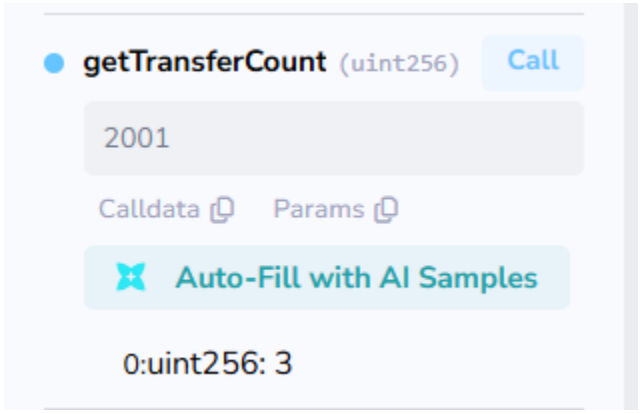

getTransferCount (uint256)
Call
2001
Calldata
Params
Auto-Fill with AI Samples
0:uint256: 1

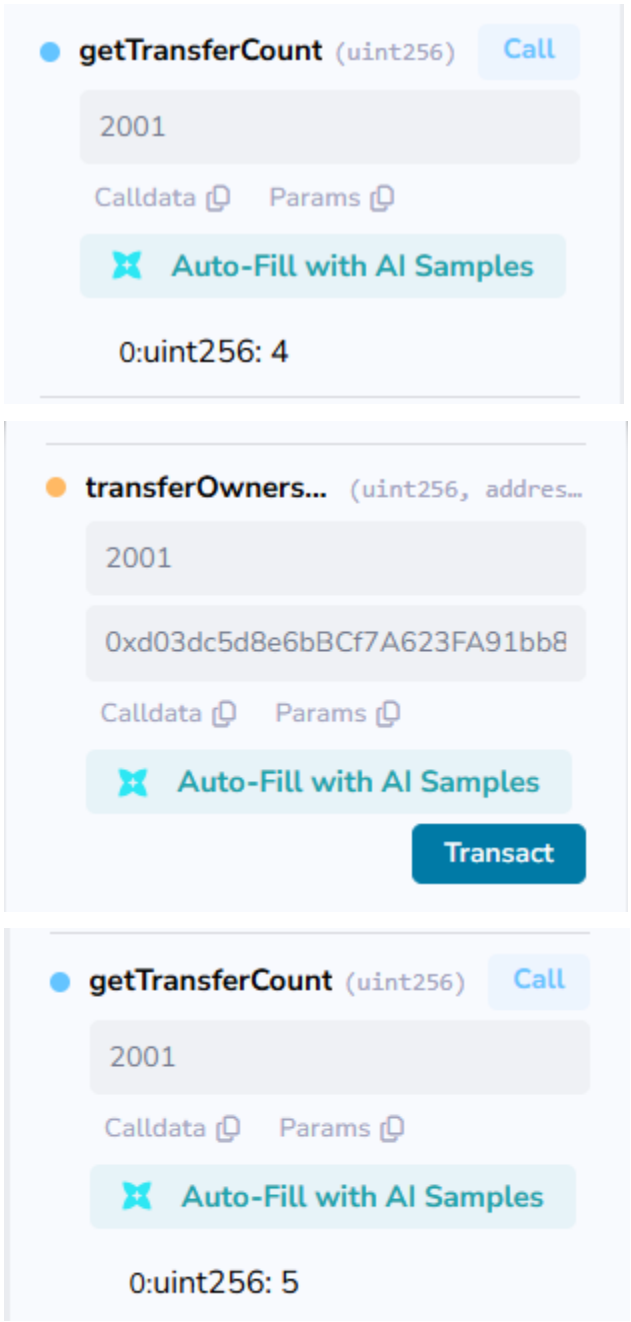

transferOwners... (uint256, addres...
2001
0xa8c4b0ECe9106ECF63662E5430
Calldata
Params
Auto-Fill with AI Samples
Transact

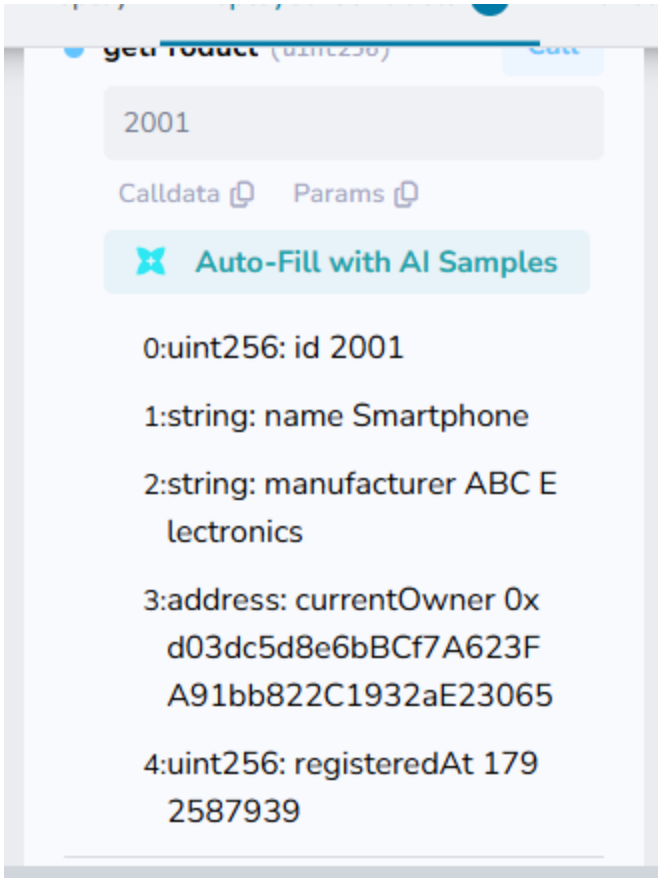

getTransferCount (uint256)
Call
2001
Calldata
Params
Auto-Fill with AI Samples
0:uint256: 2

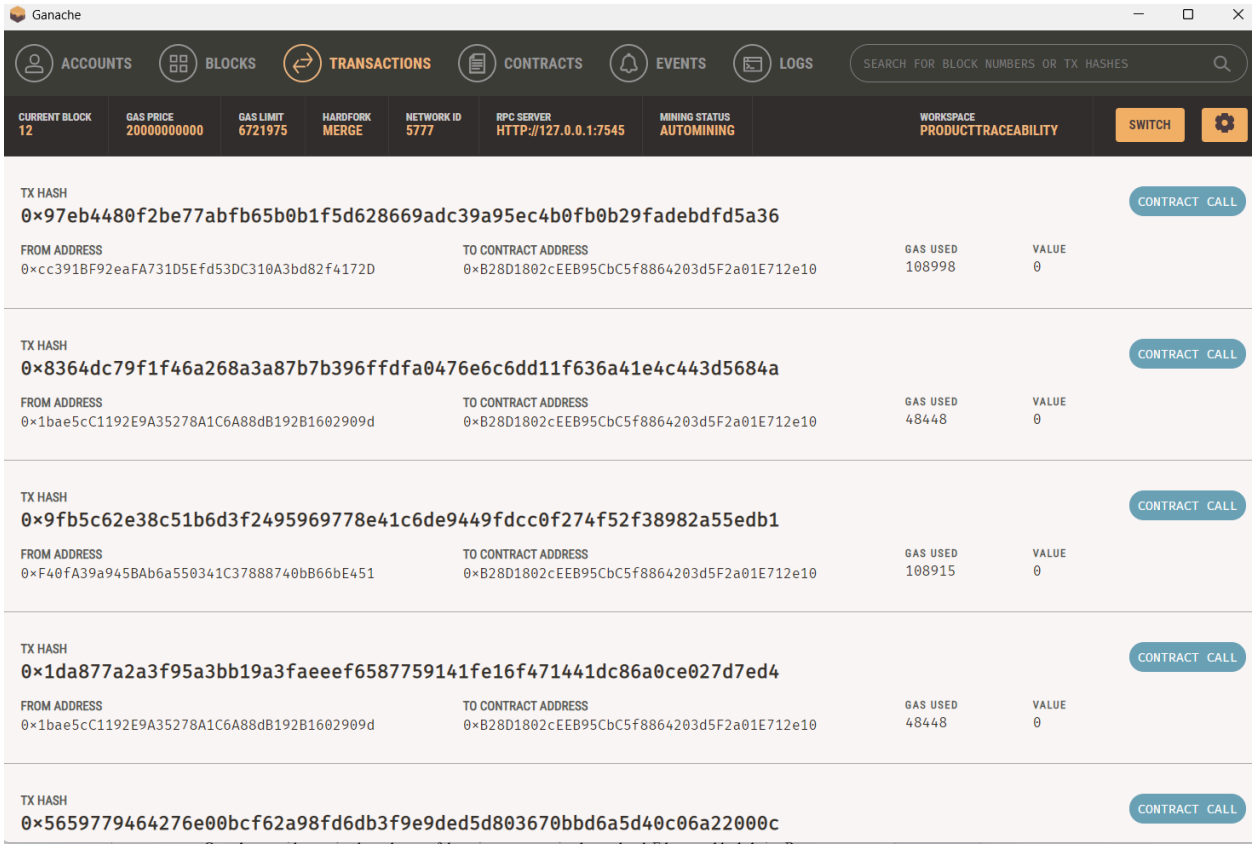

getTransfer... (uint256, ui...
Call
2001
1
Calldata
Params
Auto-Fill with AI Samples
0:address: previousOwner 0x1bae5cC1192E9A35278A1C6A88dB192B1602909d
1:address: newOwner 0xa8c4b0ECe9106ECF63662E54300CE142627d8C50


transferOwners... (uint256, addres...
2001
0xF40fA39a945BAb6a550341C37
Calldata
Params
Auto-Fill with AI Samples
Transact


getTransferCount (uint256)
Call
2001
Calldata
Params
Auto-Fill with AI Samples
0:uint256: 3


getTransferCount (uint256)
Call
2001
Calldata
Params
Auto-Fill with AI Samples
0:uint256: 4
transferOwners... (uint256, addres...
2001
0xd03dc5d8e6bBCf7A623FA91bb8
Calldata
Params
Auto-Fill with AI Samples
Transact
getTransferCount (uint256)
Call
2001
Calldata
Params
Auto-Fill with AI Samples
0:uint256: 5


2001
Calldata
Params
Auto-Fill with AI Samples
0:uint256: id 2001
1:string: name Smartphone
2:string: manufacturer ABC E lectronics
3:address: currentOwner 0x d03dc5d8e6bBCf7A623F A91bb822C1932aE23065
4:uint256: registeredAt 179 2587939



Conclusion:

The experiment was successfully performed by establishing a local Ethereum blockchain using Ganache and connecting it with MetaMask and Remix IDE. The `ProductTraceability` smart contract was compiled, deployed, and tested to register a product, assign roles to supply-chain participants, and transfer product ownership. The product details and transaction history were verified through the smart contract functions.

Thus, the experiment demonstrated how blockchain technology and Solidity smart contracts can be used to maintain a transparent, secure, and traceable record of product ownership throughout the supply chain.